

联邦式身份感知领域数据平台正式方案

文档编号: FDDP-ARCH-001

版本: Formal V9 - Engineering Adoption Edition

状态: 正式工程方案稿

日期: 2026-05-04

适用对象: 架构评审、技术立项、MVP 规划、平台治理、安全评审、PoC 实施、业务团队接入评估

核心关键词: View Plane、SDK/DX、Query Plane、Command Plane、Control Plane、Event Plane、Observability Plane、Domain Node、授权感知缓存、标准响应协议、Resource Query Descriptor、事件失效、Legacy Migration、只读闭环 MVP

文档变更摘要

Formal V9 在 V8 的基础上进一步从“工程完整”收敛到“可被业务团队采用”。本版重点补强 SDK / Developer Experience、缓存授权安全、Resource Query 细节、事件失效 DSL、安全撤销、老接口迁移、性能保护和 PoC 测试矩阵。

修正项	V8 风险	V9 处理方式
SDK / DX	Developer Portal 过轻，业务开发可能觉得平台过重	新增独立章节 SDK & Developer Experience，定义类型生成、IDE 补全、错误映射、DevTools、lint、影响分析和 Lite 模式。
缓存授权边界	Query 流程先查缓存，可能被误解为缓存绕过 Domain Node	明确 Cache hit 不是授权通过，只能复用同一授权上下文下由 Domain Node 产生的裁决结果。
resourceVersion	entityKey 是否包含 resourceVersion 不清晰	默认采用稳定 logicalEntityKey，resourceVersion 作为 Entity Record 元数据；可选 immutable version key 需维护 latest pointer。
Resource Query 示例	relation owner 与 ownerDomain / 字段 owner 概念冲突	查询示例统一改为 `expand: { owner: [...]}`。
Cursor 语义	分页在数据变更、权限变化、排序字段变化时行为未定义	新增 Cursor Contract，规定签名、绑定 filter/order/snapshot/auth、过期、篡改拒绝和 visibleCount 语义。
Event Mapping	condition 字符串可能演变为任意脚本	明确 condition 必须是受限 DSL，可静态分析、可单元测试、有限复杂度、禁止外部服务访问。
LKG 安全	LKG 保可用性，但紧急权限收紧可能被旧策略继续放行	新增 Emergency Revocation Channel，支持高敏字段 deny list、强制 policyVersion revoke、SDK 清理缓存。
旧接口迁移	真实项目需要 REST/BFF 与新平台并存	新增 Legacy API Migration Strategy，定义绞杀者迁移、灰度、回退、双读对比和 escape hatch。
性能保护	仅有复杂度和 SLO 不够工程化	新增 request coalescing、batch loading、stampede protection、per-domain concurrency、per-tenant rate limit、timeout budget。
测试矩阵	CI 阻断规则不等于可执行测试计划	新增 PoC 必跑测试矩阵，覆盖权限、缓存、cursor、partial response、LKG、Event Mapping 和 SDK。

目录

- 1. 执行摘要
- 2. 背景与问题定义
- 3. 架构原则与关键决策
- 4. 总体架构与平面职责
- 5. SDK & Developer Experience
- 6. View Plane: 页面声明式数据契约
- 7. Query Plane: 读侧查询模型
- 8. 标准返回协议与失败语义
- 9. 权限与安全裁决模型
- 10. 完整缓存模型
- 11. 事件失效模型
- 12. Control Plane 运行时治理
- 13. 性能保护与运行时防护
- 14. Observability Plane 敏感数据治理
- 15. Legacy API Migration Strategy
- 16. Command Plane 后续扩展
- 17. 特殊数据层与数据库策略
- 18. Future Extension: Cross-Site Consent & Delegation Plane
- 19. MVP 路线与验收指标
- 20. PoC 测试矩阵
- 21. 风险矩阵与反模式
- 22. 最终建议
- A. 附录: 契约与协议模板

1. 执行摘要

1.1 一句话定义

Federated Identity-Aware Domain Platform

=

SDK & Developer Experience

+ View Plane

+ Query Plane

+ Control Plane

+ Event Plane

+ Observability Plane

+ Phase 2: Command Plane

+ Future Extension: Cross-Site Consent & Delegation Plane

该平台的目标不是简单替代 REST、GraphQL 或 BFF，而是建立一套以领域、身份、权限、缓存、页面生命周期、开发者体验和可观测性为核心的数据访问范式。

前端通过页面、组件或方法声明数据需求；读侧通过受契约约束的字段、集合、聚合和关联展开获取数据；Domain Node 执行最终授权裁决；缓存只复用已经通过相同授权上下文裁决的数据；事件系统根据领域事实和 Contract Registry 生成失效计划；Control Plane 负责契约、策略、版本和运行时治理；SDK/DX 层负责让业务开发以低负担方式接入。

1.2 当前版本的核心判断

判断	结论
是否值得做	对中大型、多租户、微服务、微前端和权限复杂项目有较高实现价值。
第一版做什么	第一版只做 View + Query + SDK 的只读闭环，不包含 Command Plane。
开发者是否必须使用完整 View Manifest	不必须。小项目或试点可使用 Lite 模式，只用 `data.query()`。
Query 是否只做标量字段	不能。必须支持列表、分页、过滤、排序、搜索、聚合和关联展开。
谁做最终授权	Domain Node 是最终授权裁决者；Router 只做 fail-closed 预检查和治理拦截。
缓存是否能直接返回	只有缓存条目由 Domain Node 在相同授权上下文和版本下产生，并且 sensitivity 允许时，才可直接返回。
事件是否直接写 affectedFields	不作为长期契约。领域事件只描述业务事实，失效范围由 Contract Registry 计算。
Control Plane 不可用怎么办	Router / Domain Node 使用签名 LKG snapshot，但高敏安全收紧通过 Emergency Revocation Channel 强制生效。
老接口如何处理	通过绞杀者模式逐步迁移，旧 REST/BFF 与 FDDP 并存，可灰度、双读和回退。

1.3 当前阶段范围

当前阶段的主线是“只读闭环 MVP”。

包含：

SDK Lite 模式

definePageData()

data.query()

Field / Collection Contract

Domain Node 最终授权

标准返回协议

内存 normalized cache

受限 cursor pagination

基础 trace / DevTools

PoC 测试矩阵

不包含：

完整 Command Plane

Saga / Workflow

完整跨站第三方授权

多区域部署

全量自研数据库

所有旧接口一次性迁移

2. 背景与问题定义

2.1 原始问题

常见前端项目中，同一页面或不同页面经常为了多出一两个字段调用额外接口，后端又可能因此查询额外表、额外微服务或返回前端并不需要的数据。长期演进后会出现以下问题：

- 请求放大: 一个页面需要多次调用 REST / BFF / 微服务接口。
- 过度获取: API 返回了前端当前页面不需要的数据。
- 欠获取: 为了缺失字段新增接口或新增 BFF 拼装逻辑。
- 权限散落: 字段权限散落在 Controller、Service、BFF 和前端中。
- 缓存复用率低: 相同实体字段在不同页面无法统一复用。
- 微服务聚合复杂: 数据所有权被拆开，但前端需要的是业务视图。
- 调试黑盒: 抽象层越强，越需要 trace 和 DevTools 支撑。

2.2 目标

平台要解决的是：

业务开发声明 “需要什么数据” ；
 平台判断 “能不能读、从哪里读、缓存能不能复用、失败如何表达” ；
 Domain Node 最终裁决；
 SDK 让接入成本低于手写 BFF / hooks / cache。

2.3 非目标

平台不追求：

- 替代所有数据库。
- 替代所有 REST / RPC / GraphQL 接口。
- 第一阶段支持所有写操作。
- 让前端绕过 Domain Node 直接信任缓存。
- 让 Router 成为中心超级业务网关。
- 让领域事件依赖前端 UI 字段路径。

3. 架构原则与关键决策

3.1 核心原则

原则	说明
SDK 优先	平台能否被业务采用，关键取决于 SDK、类型、调试和渐进接入体验。
只读先行	第一阶段只做 Query 读闭环，写侧进入 Phase 2。
Domain Node 最终裁决	Router 可以拒绝请求，但不能替代 Domain Node 放行字段。
Cache hit 不是授权通过	缓存只能复用同一授权上下文下已经产生的授权裁决结果。
Contract 约束动态查询	列表、过滤、排序、搜索和聚合必须通过 Contract 白名单。
稳定实体键优先	默认采用 logicalEntityKey，resourceVersion 作为元数据。
领域事件不耦合 UI	事件只描述业务事实，失效由 Contract Registry 映射。
LKG + Emergency Revocation	LKG 保障可用性，紧急撤销保障安全性。
渐进迁移	旧 REST/BFF 与 FDDP 可长期共存，保留 escape hatch。
Observability 是敏感面	trace、audit、payload fingerprint 和拒绝原因都必须受治理。

3.2 架构决策记录

ADR	决策
ADR-001	MVP 收敛为 View + Query + SDK 只读闭环。
ADR-002	Query 模型采用 Domain Path + Resource Query Descriptor。
ADR-003	Domain Node 是最终授权裁决者。
ADR-004	缓存直接返回必须满足授权上下文、版本和敏感等级约束。
ADR-005	默认使用 stable logicalEntityKey，resourceVersion 是 Entity Record 元数据。
ADR-006	Cursor 必须签名，并绑定 filter、order、scope、contractVersion 和必要的 snapshot 信息。
ADR-007	Event Mapping condition 使用受限 DSL，禁止任意脚本。
ADR-008	Control Plane 使用 LKG，但安全收紧通过 Emergency Revocation Channel 强制覆盖。
ADR-009	SDK/DX 单独成章，并作为 PoC 验收指标。
ADR-010	Legacy API Migration 是正式架构组成，不是上线后的临时工作。

4. 总体架构与平面职责

4.1 总体结构



4.2 平面职责

Plane	核心职责	MVP 状态
SDK / DX	类型生成、IDE 补全、Lite 模式、错误映射、trace 跳转、lint、影响分析	MVP 必须包含
View Plane	页面、组件、方法级数据声明和生命周期绑定	MVP 必须包含基础版
Query Plane	字段、实体、集合、搜索、聚合、关联展开和标准响应	MVP 必须包含基础版
Control Plane	契约、策略、LKG、CI、灰度、Emergency Revocation	MVP 采用静态 + 最小 LKG
Event Plane	领域事件映射失效计划	MVP 可先手动失效，Phase 1 工程化
Observability Plane	trace、audit、redaction、DevTools	MVP 必须包含开发态 trace
Command Plane	显式业务命令、幂等、Outbox、Saga、审计	Phase 2
Cross-Site Consent	第三方字段授权、用户 grant、浏览器治理	Future Extension

5. SDK & Developer Experience

5.1 为什么 SDK / DX 必须单独成章

该平台的工程能力很强，但业务开发是否愿意使用，取决于它是否比手写 `useUser()`、REST、BFF 或 GraphQL hooks 更简单。如果 SDK 做得不好，平台会被认为是“重平台、重治理、重配置”。因此 SDK / DX 不是附属工具，而是平台能否落地的核心产品面。

5.2 SDK 目标

SDK 必须做到：

- 让业务开发能在 IDE 中看到可用域、资源、字段和类型。
- 让 `definePageData()` 在编译期发现字段不存在、filter 不合法、expand 不允许。
- 让 `data.query()` 返回类型自动推导，不需要手写 DTO。
- 让常见错误码有默认 UI 行为，不要求每个页面重复写错误分支。
- 让 traceId 一键跳转开发者工具，定位 cache hit、miss、denied、timeout、Domain Node 调用。
- 让 missing / unused field 在 lint 阶段暴露。
- 让 contract 变更能自动影响分析到页面、组件、方法和微前端应用。
- 让小项目可以使用 Lite 模式，不强制 View Manifest。

5.3 字段类型生成

Contract Registry 输出 SDK schema:

```
field: me.profile.name
type: string
nullable: false
sensitivity: private-low
ownerDomain: user
```

生成 TypeScript 类型:

```
type MeProfileFields = {
  name: string
  avatar: string | null
  email: string | null
}
```

要求:

- 类型生成必须基于 contractVersion。
- 生成结果必须包含 nullable、sensitivity、error policy、deprecated 标记。
- 删除字段必须经过 deprecation 周期，否则 CI 阻断。
- SDK 类型包应支持按 domain 拆分，避免巨型类型包拖慢 IDE。

5.4 IDE 自动补全

业务开发写:

```
const pageData = definePageData({
  me: {
    profile: ["name", "avatar"]
  }
})
```


IDE 应能补全:

```
me.profile.name
me.profile.avatar
me.profile.email
me.settings.locale
tenant.current.name
project.list
```

自动补全必须基于当前应用可见的 contract，不应暴露未发布、未授权或内部实验字段。

5.5 definePageData() 类型校验

`definePageData()` 必须在编译期和开发态校验:

- 字段是否存在。
- 字段是否 deprecated。
- Collection 是否设置分页。
- filter / orderBy 是否在白名单。
- expand 是否在 allowedExpands。
- critical 数据是否存在 required failure policy。
- 高敏字段是否被错误声明为可 prefetch 或持久化。

示例:

```
export const dashboardData = definePageData({
  critical: {
    me: { profile: ["name", "avatar"] },
    tenant: { current: ["name"] }
  },
  lazy: {
    project: {
      list: collection({ first: 20 }).select({
        fields: ["id", "name", "updatedAt"],
        expand: {
          owner: ["id", "name", "avatar"]
        }
      })
    }
  }
})
```

5.6 data.query() 返回类型推导

Lite 模式和方法级查询需要直接使用 `data.query()`:

```
const result = await data.query({
  me: { profile: ["name", "avatar"] },
  global: { config: ["appName"] }
})
```

返回类型应自动推导为:

```
type Result = {
  me: {
    profile: {
      name: string
      avatar: string | null
    }
  }
}
```

```
}
global: {
  config: {
    appName: string
  }
}
```

开发者不应手写响应 DTO。

5.7 错误码到 UI 行为的默认映射

SDK 应提供默认错误映射:

错误码	默认 UI 行为
FIELD_DENIED	对 optional 字段显示空态; 对 critical 字段进入权限错误页。
FIELD_TIMEOUT	lazy 区域显示局部重试按钮。
DOMAIN_UNAVAILABLE	显示局部服务不可用, 若允许 stale 则展示旧数据标记。
COMPLEXITY_LIMIT	开发态报错, 生产返回通用错误。
FILTER_NOT_ALLOWED	开发态提示 contract 不允许该过滤条件。
CURSOR_INVALID	重置分页并重新加载第一页。
TENANT_CONTEXT_CHANGED	清理页面私有状态并重新获取。
CACHE_UNSAFE	强制绕过缓存, 重新请求 Domain Node。

业务页面可覆盖默认映射, 但不应每个页面重复实现基础逻辑。

5.8 traceId 一键跳转开发者工具

每个响应必须包含 `meta.traceId`:

```
{
  "meta": {
    "traceId": "trace_abc",
    "requestId": "req_123"
  }
}
```

SDK DevTools 应支持:

- 通过 traceId 打开本次查询详情。
- 查看页面 manifest、query descriptor、Domain Node 调用、cache hit/miss、partial errors。
- 查看字段被拒绝的简化原因和内部审计 decisionId。
- 禁止展示字段原始值, 除非处于开发环境且字段 sensitivity 允许。

5.9 missing / unused field lint

构建期和开发态应输出:

```
missing field: 页面代码访问 data.me.profile.email, 但 pageData 未声明。
unused field: pageData 声明 tenant.current.plan, 但页面未使用。
invalid filter: project.list filter.priority 未在 contract allowedFilters 中。
unsafe cache: private-high 字段被声明为 persistent。
```

lint 应区分:

- error: 生产必须阻断。
- warning: 允许发布但需要记录。
- info: 可优化提示。

5.10 Contract 变更自动影响分析

Contract 变更时, 系统应自动输出影响范围:

字段 me.profile.avatar 变更:

- 影响页面: /dashboard, /settings/profile
- 影响组件: UserMenu, ProfileCard
- 影响方法: openInviteModal
- 影响微前端: app-shell, billing-console
- 是否破坏类型兼容: 否
- 是否需要 cacheSchemaVersion: 否

删除字段、字段类型收窄、sensitivity 提升、failure policy 变严、filter/orderBy 收窄，必须触发 CI 阻断或强制迁移计划。

5.11 Lite 模式

小项目和 PoC 不应被迫接入完整 View Manifest。SDK 必须支持 Lite 模式:

```
const result = await data.query({
  me: { profile: ["name", "avatar"] }
})
```

Lite 模式特点:

- 不要求页面 manifest。
- 仍然使用 Contract 类型、Domain Node 授权、标准返回协议和缓存安全规则。
- 不提供完整页面级 missing/unused 分析。
- 适合小项目、迁移期、低复杂页面和方法级交互。

5.12 SDK 验收指标

指标	目标
新页面接入成本	简单页面 30 分钟内完成 data.query() 或 definePageData() 接入。
类型覆盖率	100% 返回字段由 contract 生成类型覆盖。
错误处理重复代码	常见错误码由 SDK 默认处理，页面仅处理业务空态。
trace 可达性	从页面错误到 trace 详情不超过 2 次点击。
lint 有效性	missing field、unused field、unsafe persistent cache 能在本地或 CI 暴露。
Lite 模式可用性	小项目可不启用完整 View Manifest 也能使用 Query Plane。

6. View Plane: 页面声明式数据契约

6.1 定位

View Plane 负责表达“页面、组件、方法什么时候需要哪些数据”。它不负责最终授权，不替代 Query Plane，也不作为安全边界。

6.2 页面级声明

```
export const dashboardData = definePageData({
  critical: {
    me: {
      profile: ["name", "avatar"],
      permissions: ["canCreateProject"]
    },
    tenant: {
      current: ["name", "plan"]
    }
  },
  lazy: {
    project: {
      list: collection({ first: 20 }).select({
        fields: ["id", "name", "updatedAt"],
        expand: {
          owner: ["id", "name", "avatar"]
        }
      })
    },
    notification: {
      unread: ["count"]
    }
  }, {
    scope: "page",
    ssr: true,
    prefetch: "on-route-hover"
  })
```

6.3 组件级 Fragment

```
const UserMenuData = defineFragment({
  me: {
    profile: ["name", "avatar"]
  }
})
```

页面可以组合多个 fragment，View Compiler 负责去重和生成统一查询计划。

6.4 方法级声明

```
const invitePreviewData = defineMethodData({
  tenant: {
    current: ["memberCount", "memberLimit"]
  }
})
```

```
  },
  me: {
    permissions: ["canInviteMember"]
  }
}, {
  scope: "method",
  abortOnUnmount: true,
  cache: "transient"
})
```

方法级声明只适合交互触发数据，不应承载页面首屏主数据。

6.5 生命周期模型

生命周期	适用数据	缓存行为
global	公共配置和匿名可读数据	可跨页面共享，可较长 TTL。
session	登录会话级用户信息	绑定 subjectId、tenantId、permissionVersion、policyVersion。
page	页面主数据	页面卸载释放引用，不一定删除实体缓存。
component	组件局部数据	组件卸载释放引用。
method	临时交互数据	默认请求结束释放，可短暂复用。
command	写操作状态	Phase 2，绑定 commandId 和 operationId。

7. Query Plane: 读侧查询模型

7.1 定位

Query Plane 负责把 View Plane 或 ``data.query()`` 的请求转换为授权感知查询计划。它不只处理 ``me.profile.name`` 这类标量字段，还必须覆盖真实页面常用的列表、分页、过滤、排序、搜索、聚合和关联展开。

7.2 查询类型

类型	用途	示例
Scalar Field	单个字段	<code>`me.profile.name`</code>
Entity Query	指定实体详情	<code>`project.detail(id)`</code>
Collection / Connection	列表、分页	<code>`project.list(first, after)`</code>
Search	搜索和关键词查询	<code>`project.search(keyword)`</code>
Aggregate	计数、求和、分组	<code>`tenant.members.count(filter)`</code>
Relation Expansion	关联展开	<code>`project.detail(id).expand.owner`</code>

7.3 Resource Query Descriptor

为避免前端把 Query Plane 变成远程 SQL，列表、搜索、过滤和排序必须通过受 Contract 约束的 descriptor 表达。

```
data.query({
  project: {
    list: collection({
      first: 20,
      after: cursor,
      filter: {
        status: { eq: "active" },
        ownerId: { eq: "me" }
      },
      orderBy: [
        { field: "updatedAt", direction: "desc" }
      ]
    }).select({
      fields: ["id", "name", "updatedAt"],
      expand: {
        owner: ["id", "name", "avatar"]
      }
    })
  }
})
```

注意: ``ownerDomain`` 表示契约所有者，``expand.owner`` 表示项目的 owner 关联，两者必须在文档、SDK 类型和 Contract 中区分。

7.4 Collection Contract

```
resource: project.list
type: connection
ownerDomain: project
pagination:
  mode: cursor
  defaultPageSize: 20
  maxPageSize: 50
  cursor:
```

```

signed: true
expiresIn: 30m
binds:
  - resourceName
  - canonicalFilterHash
  - canonicalOrderHash
  - contractVersion
  - authContextFingerprint
  - snapshotToken
allowedFilters:
  status:
    operators: [eq, in]
  ownerId:
    operators: [eq]
    values: [me, explicit]
  updatedAt:
    operators: [gte, lte]
allowedOrderBy:
  - updatedAt
  - createdAt
  - name
allowedExpands:
  owner:
    fields: [id, name, avatar]
  members:
    maxItems: 20
count:
  visibleCount:
    mode: optional
    precision: exact_or_estimated
  totalCount:
    permission: project.admin.readTotalCount
complexity:
  base: 3
  perNode: 1
  expandCost:
    owner: 2
    members: 5

```

7.5 Cursor 语义

Cursor 必须是服务端签名的不透明令牌，客户端不得解析或伪造。

Cursor 应绑定:

- `resourceName`
- `canonicalFilterHash`
- `canonicalOrderHash`
- `pageSize`
- `contractVersion`
- `authContextFingerprint`
- `snapshotToken` 或 `readTimestamp`
- `lastSortValues`
- `direction`
- `expiresAt`

示例结构仅用于说明，实际应编码并签名：

```
{
  "resource": "project.list",
  "filterHash": "fh_abc",
  "orderHash": "oh_def",
  "lastSortValues": ["2026-05-04T10:00:00Z", "project_123"],
  "snapshotToken": "snap_001",
  "authContextFingerprint": "auth_fp_123",
  "contractVersion": "contract_v12",
  "expiresAt": "2026-05-04T10:30:00Z",
  "signature": "sig_xyz"
}
```

Cursor 行为规则：

场景	行为
Cursor 被篡改	返回 `CURSOR_INVALID`，不得降级使用。
filter/order 与 cursor 绑定不一致	返回 `CURSOR_CONTEXT_MISMATCH`，要求重新加载第一页。
contractVersion 不兼容	返回 `CURSOR_EXPIRED` 或自动重置第一页。
权限版本变化	高敏集合必须重新加载；低敏集合可按 contract 允许重置。
列表插入/删除	Cursor 基于稳定排序键和 tie-breaker；允许最终一致，但不得重复或跳过已确认边界内数据。
排序字段变化	相关 Collection Edge Index 必须失效。
snapshot 过期	返回 `CURSOR_EXPIRED`，前端重载第一页。

7.6 visibleCount 与 totalCount

默认只返回 `visibleCount`，表示当前授权上下文下可见数量。`totalCount` 可能暴露租户规模、资源存在性和权限范围，必须单独授权。

`visibleCount` 必须声明精度：

```
visibleCount:
  mode: optional
  precision: exact | estimated | unavailable
```

返回示例：

```
{
  "nodes": [],
  "pageInfo": {
    "hasNextPage": true,
    "endCursor": "cursor_abc"
  },
  "visibleCount": {
    "value": 93,
    "precision": "estimated"
  },
  "totalCount": null
}
```

7.7 Collection Edge Index

Collection Edge Index 用于缓存集合查询的排序边和分页结果。它必须绑定：

```
resourceName
canonicalFilterHash
canonicalOrderHash
authContextKey
contractVersion
```


snapshotToken/resourceVersionRange

当排序字段变化、权限变化或 filter 相关字段变化时，相关 Edge Index 必须标记 stale 或 invalidated。

7.8 查询复杂度限制

复杂度计算应考虑：

- 基础资源成本。
- pageSize。
- expand 数量和深度。
- 嵌套列表数量。
- count 精度。
- search 成本。
- domain fan-out 数量。
- 是否命中缓存。

超过限制时返回 `COMPLEXITY\_LIMIT`，不得继续部分执行未知成本查询。

8. 标准返回协议与失败语义

8.1 Query Response Envelope

所有 Query 响应必须使用统一 envelope:

```
{
  "data": {},
  "errors": [],
  "meta": {
    "requestId": "req_123",
    "traceId": "trace_abc",
    "partial": false,
    "elapsedMs": 42,
    "cache": {
      "hit": [],
      "miss": [],
      "stale": [],
      "unsafe": []
    },
    "contractVersion": "contract_v12",
    "policyVersion": "policy_v9"
  }
}
```

8.2 部分失败示例

```
{
  "data": {
    "me": {
      "profile": {
        "name": "Tom",
        "avatar": null
      }
    }
  },
  "errors": [
    {
      "path": "me.profile.avatar",
      "code": "FIELD_TIMEOUT",
      "domain": "user",
      "severity": "degraded",
      "retryable": true,
      "staleUsed": false,
      "message": "Field is temporarily unavailable."
    }
  ],
  "meta": {
    "requestId": "req_123",
    "traceId": "trace_abc",
    "partial": true,
    "elapsedMs": 83
  }
}
```

```
}
```

8.3 错误对象规范

```
{
  "path": "project.list.nodes[0].owner.avatar",
  "code": "FIELD_DENIED",
  "domain": "project",
  "severity": "denied",
  "retryable": false,
  "staleUsed": false,
  "safeMessage": "Field is not available.",
  "decisionId": "dec_123"
}
```

对客户端暴露`safeMessage`，内部审计系统可通过`decisionId` 查询详细权限原因。

8.4 基础错误码

错误码	含义
FIELD_DENIED	字段被授权拒绝。
FIELD_MASKED	字段被脱敏。
FIELD_TIMEOUT	字段所在域超时。
DOMAIN_UNAVAILABLE	域节点不可用。
CONTRACT_NOT_FOUND	契约不存在。
FIELD_NOT_REGISTERED	字段未注册。
FILTER_NOT_ALLOWED	过滤条件不在白名单。
ORDER_NOT_ALLOWED	排序字段不在白名单。
EXPAND_NOT_ALLOWED	关联展开不被允许。
PAGE_SIZE_LIMIT	分页大小超过限制。
COMPLEXITY_LIMIT	查询复杂度超过限制。
CURSOR_INVALID	Cursor 签名无效或被篡改。
CURSOR_CONTEXT_MISMATCH	Cursor 与当前 filter/order/auth 不匹配。
CURSOR_EXPIRED	Cursor 过期或快照失效。
POLICY_VERSION_EXPIRED	策略版本过期。
PERMISSION_VERSION_EXPIRED	权限版本过期。
TENANT_CONTEXT_CHANGED	租户上下文变化。
CACHE_UNSAFE	缓存条目不满足安全复用条件。
STALE_NOT_ALLOWED	当前字段不允许 stale 返回。

8.5 失败策略 Contract

```
field: billing.subscription.status
failure:
  mode: stale-if-error
  maxStale: 5m
  retryable: true
  requiredForPage: false
```

可选模式:

模式	行为
required	失败导致所在 critical group 失败。
optional	返回 null + error。
stale-if-error	上游失败时可返回旧数据，但必须满足 maxStale 和授权版本一致。
null-if-denied	无权限时返回 null。
error-if-denied	无权限时返回 FIELD_DENIED。
masked	返回脱敏值。

8.6 页面级失败策略

Scope	行为
-------	----

critical	失败进入页面级 fallback 或 error boundary。
lazy	局部错误，不阻塞页面主渲染。
method	返回方法错误，不污染页面主状态。
prefetch	静默失败并记录 trace。

9. 权限与安全裁决模型

9.1 权限上下文

```
{
  "subjectId": "user_123",
  "tenantId": "tenant_abc",
  "roles": ["member"],
  "permissionVersion": "perm_v17",
  "policyVersion": "policy_v9",
  "clientId": "first_party",
  "grantId": null,
  "purpose": "default",
  "sessionId": "sess_123"
}
```

9.2 Router 与 Domain Node 边界

硬规则:

> Domain Node 是最终授权裁决者。Router 可以拒绝请求，但不能替代 Domain Node 放行敏感字段、集合、关联或聚合。

Router 负责:

- 校验 token、tenant、session、origin 基础边界。
- 检查 query 是否命中已注册 contract。
- 执行复杂度限制、限流、熔断和黑名单拦截。
- 使用 LKG snapshot 做 fail-closed 预检查。
- 在无法确认安全时拒绝请求。

Domain Node 负责:

- 对本域字段、集合、聚合、关联展开做最终授权裁决。
- 应用权限过滤后的 filter。
- 决定返回 data、null、masked 或 error。
- 生成可缓存的授权裁决元数据。

Policy Engine 是决策辅助组件，不是业务 owner。它可被 Router 和 Domain Node 调用，但最终责任由 Domain Node 承担。

9.3 Cache hit 不是授权通过

缓存读取必须遵守以下规则:

> Cache hit 不是授权通过。Cache hit 只能复用已有授权裁决结果。

只有当缓存条目满足以下条件时，SDK / Router 才可直接返回:

- 缓存条目由 Domain Node 产生。
- `authContextKey` 完全匹配或满足 Contract 声明的安全兼容规则。
- `permissionVersion` 匹配。
- `policyVersion` 匹配。
- `contractVersion` 兼容。
- `resourceVersion` 未失效。
- `decisionFingerprint` 匹配。

- 字段 `sensitivity` 允许当前缓存层级返回。
- 租户上下文未变化。
- 没有 emergency deny / revoke 命中。

以下情况必须重新裁决或直接标记 unsafe:

- 高敏字段。
- 权限降级后字段。
- policyVersion 变化后字段。
- grant/purpose 变化后字段。
- 租户切换后 private/tenant 字段。
- LKG 过期且字段 sensitivity 为 private-high 或 secret。

9.4 裁决顺序

1. Router 基础身份、租户、契约和复杂度预检查。
2. Router 检查 emergency deny / revoke channel。
3. Router 生成 domain-specific subquery。
4. Domain Node 读取必要资源和权限关系。
5. Domain Node 执行字段级、行级、关系级、聚合级裁决。
6. Domain Node 返回 data + errors + decision metadata。
7. Cache Runtime 按 decision metadata 写入允许层级。

10. 完整缓存模型

10.1 设计目标

缓存模型必须同时满足：

- 减少重复请求。
- 不绕过最终授权裁决。
- 支持实体字段复用。
- 支持集合分页索引。
- 支持事件失效。
- 支持租户切换和权限降级清理。
- 支持敏感字段持久化限制。
- 支持 SDK DevTools 可解释。

10.2 缓存层级

层级	位置	用途
L0	Request Scope	单次请求去重、DataLoader、批处理。
L1	Browser Memory / SDK Runtime	页面与会话级实体缓存。
L2	IndexedDB / Local Persistent	仅低敏或显式允许数据。
L3	Router / Edge Cache	public、tenant-low 或可共享的 query result。
L4	Domain Node / Service Cache	域内实体、聚合、上游服务结果。

10.3 缓存对象类型

类型	作用
Entity Record	实体基础记录和资源版本。
Field Slot	单个字段值、状态、版本、sensitivity、decision metadata。
Query Result Index	query descriptor 到实体/字段集合的索引。
Collection Edge Index	集合分页边和 cursor 关联。
Aggregate Cache	visibleCount、sum、group 等聚合缓存。
Reference Tracker	页面、组件、方法对字段/资源的引用。
Invalidation Journal	失效事件处理记录，保证幂等和可追踪。
Negative Cache	可选拒绝结果缓存，必须绑定 policyVersion 和短 TTL。

10.4 Entity Record 与 Field Slot

```
{
  "logicalEntityKey": "user:UserProfile:user_123:tenant_abc",
  "resourceVersion": "rv_18",
  "sourceVersion": "etag_abc",
  "fields": {
    "displayName": {
      "valueRef": "...",
      "state": "fresh",
      "sensitivity": "private-low",
      "resourceVersionAtWrite": "rv_18",
      "permissionVersion": "perm_v17",
      "policyVersion": "policy_v9",
      "contractVersion": "contract_v12",
      "decisionFingerprint": "dec_fp_123",
      "expiresAt": "2026-05-04T10:10:00Z"
    }
  }
}
```

```
}
}
```

10.5 Key 模型: logicalEntityKey 与 resourceVersion

默认采用方式 B: 稳定实体键 + 资源版本作为元数据。

```
logicalEntityKey =
  domain
  + entityType
  + entityId
  + tenantId
```

示例:

```
user:UserProfile:user_123:tenant_abc
```

`resourceVersion` 存储在 Entity Record 和 Field Slot 元数据中, 不参与 logicalEntityKey。这样读取时可以先根据稳定键找到缓存记录, 再判断版本是否仍可用。

可选方式 A: immutable entityVersionKey。

```
entityVersionKey = logicalEntityKey + resourceVersion
logicalEntityKey -> latest entityVersionKey
```

如果采用方式 A, 必须维护稳定指针 `logicalEntityKey -> latest entityVersionKey`, 否则读取时无法在不知道最新 `resourceVersion` 的情况下命中缓存。

本方案默认采用方式 B。

10.6 Auth Context Key

```
authContextKey =
  subjectId
  + tenantId
  + permissionVersion
  + policyVersion
  + roleHash
  + clientId
  + grantId
  + purpose
  + sessionSecurityLevel
```

示例:

```
user_123:tenant_abc:perm_v17:policy_v9:role_ab12:first_party:none:default:mfa_low
```

10.7 Field Result Key

```
fieldResultKey =
  logicalEntityKey
  + fieldPathOrFieldSetHash
  + authContextKey
  + contractVersion
  + cacheSchemaVersion
  + resourceVersionAtDecision
```

注意: `resourceVersionAtDecision` 是裁决和写入时的版本, 用于安全校验, 不是 primary key 的唯一组成。

10.8 Collection Query Key

```
collectionKey =
```



```
resourceName
+ canonicalFilterHash
+ canonicalOrderHash
+ pageCursorFingerprint
+ pageSize
+ authContextKey
+ contractVersion
+ snapshotToken
```

10.9 敏感等级与持久化规则

等级	示例	默认缓存规则
public	appName、logo	可 L2/L3/L4，TTL 可较长。
tenant-low	租户名称、公开项目名	可 L2/L3/L4，但必须 tenant namespace 隔离。
private-low	昵称、头像、语言偏好	可 L1，可选 L2，绑定 subject、permissionVersion、policyVersion。
private-high	邮箱、电话、账单状态	默认 L1 短 TTL，L2 禁止，stale 需显式允许。
secret	MFA、token、密钥、安全问题	默认不缓存或仅 L0。
permission-derived	权限、菜单、按钮能力	必须绑定 permissionVersion 和 policyVersion。
third-party-bound	grant/purpose 绑定数据	必须绑定 clientId、grantId、purpose。

L2 IndexedDB 默认不存储 private-high、secret 和 permission-derived 数据。任何例外必须通过安全评审并写入 Contract。

10.10 缓存状态机

```
fresh -> stale -> refreshing -> fresh
fresh -> invalidated -> evicted
fresh -> unsafe -> evicted
stale -> expired -> evicted
refreshing -> failed -> stale-if-error or error
```

状态	含义
fresh	可在安全条件满足时复用。
stale	已过期但可能允许 stale-if-error。
refreshing	正在后台刷新。
invalidated	被事件或版本标记失效。
unsafe	授权上下文、策略、租户或存储规则不再安全。
evicted	已删除。
denied-cached	可选拒绝结果缓存，必须短 TTL 并绑定 policyVersion。

10.11 缓存读取算法

- 1. 标准化 query descriptor。
- 2. 计算 authContextKey。
- 3. 查询 logicalEntityKey / collectionKey。
- 4. 校验 tenantId、subjectId、permissionVersion、policyVersion、contractVersion。
- 5. 校验 decisionFingerprint 和 Domain Node 裁决来源。
- 6. 校验 resourceVersion / sourceVersion。
- 7. 校验 sensitivity 是否允许当前缓存层级返回。
- 8. 校验 emergency deny / revoke 是否命中。
- 9. 判断 field slot 是否 fresh。
- 10. 如果 stale，检查 failure policy 是否允许 stale-if-error。
- 11. 如果 unsafe、invalidated 或 version mismatch，必须重新请求 Domain Node 或 fail-closed。
- 12. 返回 hit、miss、stale、unsafe 和 denied-cached 集合。

10.12 缓存写入算法

- 1. Domain Node 返回 data + version + sensitivity + cache policy + decision metadata。

2. Router / SDK 校验返回字段与 contract 一致。
3. 根据 sensitivity 决定可写入缓存层级。
4. 更新 Entity Record 和 Field Slot。
5. 更新 Query Result Index、Collection Edge Index 或 Aggregate Cache。
6. 建立 View Reference。
7. 写入 Invalidation Journal 关联信息。
8. 记录 cache written trace, 禁止记录字段值。

10.13 租户切换清理

租户切换时必须:

- 立即清理当前 tenant 的 page / component / method scope 引用。
- 清理 private cache namespace。
- 清理 tenant-scoped cache 或切换到隔离 namespace。
- public cache 可保留。
- 触发 `TENANT\_CONTEXT\_CHANGED`，页面重新加载 critical 数据。

10.14 权限降级与策略更新

收到以下事件时:

```
authz.permissionVersion.changed
policy.version.changed
tenant.role.changed
emergency.policy.revoked
```

必须:

- 清理 subject private cache。
- 清理 permission-derived cache。
- 标记旧 permissionVersion / policyVersion 的字段为 unsafe。
- private-high 和 secret 不允许 stale-if-error。
- SDK DevTools 显示缓存因安全上下文变化失效。

10.15 Cache Stampede Protection

热点缓存过期时必须避免击穿:

- 对同一 `collectionKey` / `fieldResultKey` 做 singleflight。
- 允许 stale-while-refresh, 但仅限 Contract 显式允许且敏感等级安全。
- 对 private-high 不启用 stale-while-refresh。
- 对高并发 public 数据启用 jitter TTL 和后台刷新。

11. 事件失效模型

11.1 核心原则

长期契约中，领域事件不得直接携带 UI 字段路径。领域事件只描述业务事实，失效范围由 Contract Registry 根据 Event Mapping 计算。

不推荐长期使用：

```
{
  "event": "user.profile.updated",
  "affectedFields": ["me.profile.name", "user.public.name"]
}
```

推荐：

```
{
  "eventId": "evt_123",
  "eventType": "user.profile.updated",
  "aggregateType": "UserProfile",
  "aggregateId": "user_123",
  "tenantId": "tenant_abc",
  "aggregateVersion": 18,
  "changedAttributes": ["displayName", "avatarFileId"],
  "occurredAt": "2026-05-04T10:00:00Z"
}
```

11.2 Event Mapping Contract

```
eventType: user.profile.updated
aggregateType: UserProfile
ownerDomain: user
mappings:
  - targetType: entity
    entityType: UserProfile
    entityId: "${aggregateId}"
    fields:
      - displayName
      - avatar
  - targetType: path
    path: me.profile.*
    condition:
      op: eq
      left: subjectId
      right: aggregateId
  - targetType: collection
    resource: tenant.members.list
    condition:
      op: intersects
      left: changedAttributes
      right: [displayName, avatarFileId]
  - targetType: aggregate
    resource: tenant.members.count
    condition:
      op: const
```

value: false

11.3 Condition DSL 规则

Event Mapping condition 必须使用受限 DSL，不允许任意字符串脚本。

允许能力:

- 布尔操作: `and`、`or`、`not`。
- 比较操作: `eq`、`neq`、`in`、`contains`、`intersects`。
- 常量: string、number、boolean、array。
- 可引用字段: event envelope 中的白名单字段，如 `aggregateId`、`tenantId`、`changedAttributes`、`aggregateVersion`。
- 可引用上下文: cache index metadata 中的白名单字段，如 `entityId`、`tenantId`、`subjectId`、`resourceName`。

禁止能力:

- 任意 JS / Python / Lua 脚本。
- 网络请求或外部服务访问。
- 文件访问。
- 时间依赖随机逻辑。
- 非确定性函数。
- 无界循环或递归。

工程要求:

- DSL 必须可静态分析。
- DSL 必须可单元测试。
- 每条 mapping 必须有复杂度上限。
- Mapping 变更必须执行 replay / dry-run。
- Mapping 输出必须写入 Invalidation Journal。

11.4 Invalidation Plan

Invalidation Resolver 根据领域事件和 Mapping Contract 生成计划:

```
{
  "planId": "inv_123",
  "sourceEventId": "evt_123",
  "tenantId": "tenant_abc",
  "targets": [
    {
      "type": "entity",
      "entityType": "UserProfile",
      "entityId": "user_123",
      "fields": ["displayName", "avatar"],
      "mode": "mark-stale"
    },
    {
      "type": "collection",
      "resource": "tenant.members.list",
      "filterMatcherId": "fm_abc",
      "mode": "invalidate-index"
    },
    {
      "type": "view-reference",
```

```
    "scopeMatcherId": "sm_depends_me_profile",
    "mode": "notify-runtime"
  }
],
"generatedAt": "2026-05-04T10:00:01Z"
}
```

11.5 失效目标类型

目标	用途
entity	实体或字段失效。
field-slot	单字段 slot 失效。
collection	列表边索引失效。
aggregate	聚合缓存失效。
query-result	某个 query descriptor 结果失效。
view-reference	通知当前页面/组件依赖刷新。
auth-context	权限上下文相关缓存失效。

11.6 失效失败安全策略

场景	行为
Event Mapping 缺失	public 可短 TTL stale; private-high / secret fail-closed 或强制 unsafe。
DSL 执行超复杂度	mapping 失败，相关域 high sensitivity 缓存 unsafe。
Invalidation Journal 不可写	暂停写入新缓存，触发告警。
事件乱序	根据 aggregateVersion 丢弃旧事件或等待补偿。
事件重复	通过 eventId 幂等处理。

11.7 Command Response 中的 hints

Phase 2 Command 可返回 invalidation hints，但它们只能作为优化提示，不是长期权威契约。

```
{
  "commandId": "cmd_123",
  "events": ["user.profile.updated"],
  "invalidationHints": [
    { "type": "entity", "entityType": "UserProfile", "entityId": "user_123" }
  ]
}
```

12. Control Plane 运行时治理

12.1 核心模块

模块	职责
Domain Registry	域、owner、endpoint、SLA、版本。
Contract Registry	Field、Collection、Command、Event Mapping 契约。
Policy Registry	全局策略、域策略、敏感字段策略。
SDK Schema Registry	类型生成、字段补全、版本兼容。
View Manifest Registry	页面、组件、方法数据声明。
Cache Governance	cache scope、TTL、sensitivity、持久化策略。
Event Schema Registry	事件 schema、版本、兼容性。
Compatibility Checker	契约变更和页面依赖兼容性检查。
Developer Portal	字段、资源、事件、trace、权限和使用方查询。
Emergency Revocation Channel	高优先级安全撤销和 deny list。

12.2 Last-Known-Good Snapshot

Router 和 Domain Node 运行时不应强依赖 Control Plane 实时可用。它们使用签名的 LKG snapshot。

```
{
  "snapshotId": "lkg_20260504_001",
  "contractVersion": "contract_v12",
  "policyVersion": "policy_v9",
  "generatedAt": "2026-05-04T09:00:00Z",
  "expiresAt": "2026-05-04T13:00:00Z",
  "signature": "sig_abc"
}
```

Registry 不可用时:

状态	行为
LKG 未过期	继续服务已知 contract，禁止新字段、新策略生效。
LKG 过期	public 低风险字段可短期服务；private-high 和 unknown 请求 fail-closed。
LKG 签名无效	fail-closed。
Domain endpoint registry 不可用	使用本地 endpoint cache，并触发降级告警。

12.3 Emergency Revocation Channel

LKG 解决可用性，但不能阻止紧急权限收紧被旧 LKG 继续放行。因此必须新增高优先级安全通道。

Emergency Revocation Channel 支持:

- 高敏字段 emergency deny list。
- 强制 `policyVersion` revoke。
- 强制 `contractVersion` revoke。
- 强制某个 `fieldId`、`domain`、`tenantId`、`subjectId`、`grantId` 失效。
- Router 本地 LKG 立即标记过期或部分失效。
- Domain Node 对 private-high / secret 请求 fail-closed。
- SDK 清理相关 private cache。
- 记录安全审计。

示例:

```
{
  "revocationId": "rev_001",
  "type": "field-deny",
  "fieldIds": ["fld_billing_invoice_latest"],
}
```

```

"minPolicyVersion": "policy_v11",
"appliesTo": {
  "tenants": ["tenant_abc"],
  "sensitivity": ["private-high", "secret"]
},
"action": "fail-closed",
"issuedAt": "2026-05-04T10:00:00Z",
"signature": "sig_rev"
}

```

执行顺序:

1. Router 接收 emergency revocation。
2. 校验签名和版本。
3. 更新本地 deny/revoke cache。
4. 标记相关缓存 unsafe。
5. 通知 SDK 清理 private cache。
6. Domain Node 对命中字段 fail-closed。
7. 写入安全审计。

12.4 策略发布流程

```

draft
-> CI validation
-> shadow evaluation
-> canary 1%
-> canary 10%
-> active
-> rollback-capable

```

Shadow evaluation 表示新策略只记录“如果启用会允许或拒绝什么”，不影响线上结果。

12.5 Contract CI 阻断规则

以下情况必须阻断发布:

- 字段 owner 不唯一。
- 删除字段没有 deprecation 周期。
- 字段类型不兼容。
- 高敏字段允许 IndexedDB 持久化但未安全评审。
- Collection 未声明 maxPageSize。
- filter、orderBy、expand 未白名单。
- Cursor 未签名或未绑定 filter/order/auth。
- Event Mapping 缺失或 DSL 不可静态分析。
- policyVersion 未递增。
- contractVersion 未递增。
- 破坏现有页面 manifest 且没有迁移计划。
- Domain Node 未实现最终授权裁决接口。

13. 性能保护与运行时防护

13.1 Request Coalescing

同一时间窗口内相同 query descriptor、authContextKey 和 contractVersion 的请求应合并为一个上游请求。

适用:

- 多组件同时请求同一字段。
- 路由切换和预取同时发生。
- 多个微前端同时启动。

13.2 Batch Loading

Domain Node 内部必须使用 batch loading 避免 N+1。

要求:

- 同一 request scope 内按 entityType / relation / source 分组批量加载。
- relation expansion 默认批量请求。
- 对集合每个节点展开 owner、avatar、permission 时不得逐行请求。

13.3 Cache Stampede Protection

热点 public / tenant-low 缓存过期时应:

- 使用 singleflight。
- 使用 jitter TTL。
- 支持 stale-while-refresh。
- 设置刷新并发上限。
- private-high 不允许通过 stale-while-refresh 绕过重新裁决。

13.4 Concurrency 与 Rate Limit

限制	说明
per-domain concurrency limit	防止单个 domain 被聚合查询压垮。
per-tenant rate limit	防止某租户或恶意客户端放大请求。
per-subject rate limit	防止单用户或会话异常访问。
per-client rate limit	为未来第三方授权预留。
collection page size limit	防止大页查询。
expansion depth limit	防止关联展开爆炸。

13.5 Timeout Budget

页面应有总预算和 domain 子预算。

示例:

```
Dashboard critical budget: 800ms
  user domain: 150ms
  tenant domain: 150ms
  project domain: 300ms
  notification domain: 120ms
  router overhead: 80ms
```

预算耗尽策略:

- critical required 字段失败: 页面进入 fallback。

- lazy optional 字段失败: 局部错误。
- stale-if-error 字段: 在安全条件满足时返回 stale。
- private-high 字段: 默认 fail-closed。

13.6 Fallback 优先级

```
public stale
> tenant-low stale
> private-low stale
> permission-derived revalidate
> private-high fail-closed
> secret fail-closed
```

13.7 SLO 指标

指标	MVP 目标
Router p95 overhead	< 30ms, 不含上游 Domain Node。
Dashboard critical p95	< 800ms。
cache safe-hit ratio	PoC 页面 > 50%。
Domain timeout fallback correctness	100% 按 failure policy 行为。
complexity rejected queries	100% 有错误码和 trace。
request coalescing effectiveness	同一 tick 重复 query 合并率 > 80%。

14. Observability Plane 敏感数据治理

14.1 核心原则

Observability 本身是敏感数据面。Trace、Audit、payload fingerprint、字段路径、权限拒绝原因和 actor 信息都必须受访问控制、脱敏、保留期和审计治理。

14.2 Trace 不记录原始值

禁止记录:

- 字段值。
- 原始 payload。
- email、phone、token、cookie。
- 完整请求体和响应体。
- 未脱敏权限拒绝原因。

14.3 字段路径脱敏

生产 trace 默认记录:

```
{
  "fieldId": "fld_8f21",
  "domain": "billing",
  "sensitivity": "private-high",
  "status": "denied"
}
```

完整字段路径只允许在开发环境或受控安全审计环境查看。

14.4 HMAC payload fingerprint

payload 指纹必须使用 HMAC:

```
payloadFingerprint = HMAC(secret, canonicalPayload)
```

禁止对低熵 payload 使用普通 SHA256 暴露可反推指纹。

14.5 Trace 与 Audit 分离

类型	用途	保留期	访问控制
Trace	性能、调试、链路分析	短	工程团队受控访问，强脱敏。
Audit	安全、合规、追责	长	安全/合规角色访问，访问本身被审计。

15. Legacy API Migration Strategy

15.1 定位

真实项目不可能一次性从 REST/BFF 迁移到 FDDP。迁移策略必须成为正式架构的一部分，而不是上线后的临时工作。

目标是：

旧接口可继续运行；
新页面优先接入 FDDP；
高频旧页面逐步改造；
失败可回退；
关键接口可保留 escape hatch。

15.2 存模式

模式	说明
Legacy REST/BFF	继续服务旧页面和低价值接口。
FDDP Adapter	将旧接口包装为 Field / Collection Contract 的数据源。
Native Domain Node	新领域直接按 FDDP contract 实现。
Escape Hatch	对复杂写操作、低价值临时接口、极端性能路径保留旧接口。

15.3 旧接口拆解方法

一个旧接口：

GET /api/dashboard

可能拆成：

me.profile.name
me.profile.avatar
tenant.current.name
project.list(first: 20)
notification.unread.count

拆解步骤：

1. 分析接口返回字段。
2. 标记每个字段的 ownerDomain。
3. 标记 sensitivity、cache scope、权限规则。
4. 将列表部分拆为 Collection Contract。
5. 将聚合字段拆为 Aggregate Contract。
6. 定义 fallback 到旧接口的条件。
7. 生成双读对比计划。

15.4 灰度与回退

迁移流程：

legacy only
-> adapter shadow read
-> dual read compare
-> 1% FDDP read
-> 10% FDDP read

-> new pages FDDP default
-> old API deprecate

回退策略:

- FDDP query error rate 超阈值，页面回退 legacy API。
- Contract 不兼容时阻断发布。
- Domain Node 超时可走 legacy fallback，但必须记录 trace。
- 安全相关字段不得因 fallback 绕过 Domain Node 授权。

15.5 双读对比

双读期间比较:

- 字段值一致性。
- 权限过滤结果。
- visibleCount 差异。
- 排序和分页稳定性。
- 响应耗时。
- cache hit ratio。

差异必须分类:

expected difference: 新平台做了更严格权限过滤。
source mismatch: 旧接口和 Domain Node 数据源不同步。
bug: FDDP contract 或 resolver 错误。
security issue: FDDP 放行了旧接口不应暴露字段。

15.6 永不迁移或暂缓迁移的接口

以下接口可保留 escape hatch:

- 极端性能敏感、已深度优化的接口。
- 一次性内部工具接口。
- 复杂写操作接口。
- 低价值、短生命周期功能。
- 第三方供应商回调。

16. Command Plane 后续扩展

16.1 定位

Command Plane 是 Phase 2 能力。写侧不采用字段自由赋值，而采用显式领域命令。

不推荐:

```
data.me.profile.name = "Tom"
```

推荐:

```
await command.user.profile.update({
  displayName: "Tom",
  avatarFileId: "file_123"
}, {
  idempotencyKey: "cmd_123",
  expectedVersion: 18
})
```

16.2 与 Query Plane 的关系

- Command Plane 负责业务动作、幂等、事务、Outbox、审计。
- Query Plane 负责读取、缓存、聚合、失败语义。
- Command 成功后发布领域事件。
- Event Plane 根据 Contract Mapping 生成 Invalidation Plan。
- Query Cache 根据 Invalidation Plan 失效。

16.3 Phase 2 最小命令

Phase 2 可先实现:

```
user.profile.update
tenant.member.invite
tenant.member.remove
global.config.update
```

必须包含:

- 命令权限。
- 幂等 key。
- 乐观并发。
- 本地事务。
- Outbox。
- 审计。
- 读侧缓存失效。

17. 特殊数据层与数据库策略

17.1 定位

该平台不应第一阶段自研数据库。更合理的方式是建立 Domain Metadata Layer 和 Domain Data Engine，使现有数据库、REST、GraphQL、搜索服务和缓存系统表现为统一领域数据层。

17.2 推荐组合

PostgreSQL / MySQL / Existing Services: 真实业务数据源

Redis / Domain Cache: 服务端缓存

Browser Memory / IndexedDB: 前端安全缓存

Contract Registry: 字段、集合、事件、策略元数据

Domain Node: 领域查询和最终授权裁决

17.3 什么时候考虑特殊存储

只有在以下条件满足时才考虑专用存储:

- 字段状态极多，普通缓存难以管理。
- 授权裁决和缓存版本强耦合。
- 需要边缘 / 离线 / 多端同步。
- Contract Registry 成为核心运行时元数据服务。
- 读侧 projection 和 event replay 成为平台核心能力。

18. Future Extension: Cross-Site Consent & Delegation Plane

18.1 定位

跨站授权与第三方委托是未来拓展层，不进入当前 MVP。它的目标是在平台成熟后，将字段级数据治理扩展到第三方应用和跨站访问场景。

18.2 当前只预留扩展点

当前阶段只在模型中预留：

- `clientId`
- `grantId`
- `origin`
- `purpose`
- `thirdParty.shareable`
- `thirdParty.allowedPurposes`
- `thirdParty.retention`

18.3 未来授权交集模型

实际可访问字段

=

平台允许字段

∩ 第三方申请字段

∩ 用户授权字段

∩ 浏览器 / Origin 允许

∩ 运行时权限裁决

19. MVP 路线与验收指标

19.1 MVP-0: 只读闭环

包含:

- SDK Lite 模式。
- `definePageData()` 基础能力。
- `data.query()` 类型推导。
- me / global / tenant / project 基础域。
- Scalar Field、Entity Query、一个 Collection Query。
- 静态 Field / Collection Contract。
- Domain Node 最终授权裁决。
- 内存 normalized cache。
- 标准 Response Envelope。
- 开发态 trace 与 DevTools。

不包含:

- Command Plane。
- Outbox / Saga。
- 完整 Event Bus。
- 完整 Control Plane。
- 跨站授权。
- 高敏 IndexedDB 缓存。

19.2 MVP-0 验证页面

Dashboard:

```
me.profile.name
me.profile.avatar
tenant.current.name
project.list(first: 20, orderBy: updatedAt desc)
notification.unread.count
```

19.3 MVP-0 验收指标

指标	目标
重复字段缓存命中	页面二次进入时 me/global 字段命中。
权限拒绝	无权限字段由 Domain Node 拒绝，不由 Router 或前端隐藏。
列表查询	project.list 不退回定制接口。
Cursor 安全	篡改 cursor 返回 `CURSOR_INVALID`。
partial response	notification 超时时页面仍可渲染 critical 数据。
SDK 类型	data.query 返回类型自动推导。
trace	一键从页面错误定位到字段和 Domain Node 调用。
tenant 切换	private/tenant cache 不串租户。

19.4 MVP-1: 读侧工程化

新增:

- Contract Registry。
- Event Mapping + Invalidation Plan。

- LKG snapshot。
- Emergency Revocation Channel。
- SDK lint 与影响分析。
- request coalescing / batch loading / stampede protection。
- Legacy API dual read compare。

19.5 MVP-2: Command Plane

新增:

- 显式领域命令。
- 幂等和乐观并发。
- Outbox。
- 审计。
- 命令事件驱动读侧失效。

20. PoC 测试矩阵

20.1 权限测试

测试	期望
字段未注册	默认拒绝，返回 FIELD_NOT_REGISTERED。
Router 放行但 Domain Node 拒绝	最终返回 FIELD_DENIED。
用户无 totalCount 权限	返回 visibleCount，totalCount 为 null 或 error。
聚合存在性泄露	无权限不得通过 count 暴露资源存在。

20.2 缓存安全测试

测试	期望
权限降级后旧缓存	标记 unsafe，不可返回。
policyVersion 变化	旧缓存不可直接返回。
租户切换	private/tenant cache 不串。
高敏字段 IndexedDB	不写入 L2。
Cache hit 未匹配 authContextKey	返回 CACHE_UNSAFE 或重新裁决。
emergency deny 命中	本地缓存立即 unsafe。

20.3 Cursor 与 Collection 测试

测试	期望
Cursor 被篡改	CURSOR_INVALID。
filter/order 变更复用旧 cursor	CURSOR_CONTEXT_MISMATCH。
contractVersion 不兼容	CURSOR_EXPIRED。
排序字段变化	Collection Edge Index 失效。
权限变化后继续分页	高敏集合重新加载或 fail-closed。

20.4 Event Mapping 测试

测试	期望
Mapping 缺失	private-high fail-closed 或缓存 unsafe。
DSL 包含脚本	CI 阻断。
DSL 访问外部服务	CI 阻断。
Mapping replay	输出可比较 Invalidation Plan。
事件重复	Invalidation Journal 幂等处理。

20.5 Control Plane 测试

测试	期望
Registry 不可用且 LKG 未过期	已知低风险 contract 可继续服务。
LKG 过期	private-high fail-closed。
LKG 签名无效	fail-closed。
Emergency Revocation	立即覆盖 LKG，相关字段拒绝。
策略 shadow	不影响线上结果，仅记录差异。

20.6 SDK / DX 测试

测试	期望
definePageData 拼错字段	TypeScript 或 lint 报错。
data.query 返回类型	自动推导，无需手写 DTO。
unused field	lint warning。
missing field	lint error 或 dev warning。
traceId 跳转	能打开 DevTools 查询详情。
Lite 模式	不使用 View Manifest 仍可安全查询。

20.7 Partial Response 测试

测试	期望
lazy domain 超时	页面 critical 正常渲染，lazy 区域显示局部错误。
critical required 失败	页面进入 fallback。
stale-if-error 允许	在版本和敏感等级安全时返回 stale。
private-high stale	默认不返回，fail-closed。

21. 风险矩阵与反模式

风险	后果	缓解
MVP 偏大	做不完、闭环不清	第一版只读闭环，Command 延后。
SDK 过重	业务团队不接入	Lite 模式、类型生成、DevTools、错误映射。
Router 中心化	变成超级网关	Domain Node 拥有最终授权和领域执行。
缓存绕过授权	旧权限读到新数据	Cache hit 只复用相同授权上下文的裁决结果。
Cursor 不稳定	分页重复、跳页、伪造	签名 cursor，绑定 filter/order/auth/snapshot。
Event Mapping 任意脚本	安全和性能风险	受限 DSL，静态分析，dry-run。
LKG 旧策略放行	紧急安全事故	Emergency Revocation Channel。
旧接口迁移失败	项目无法落地	并存、灰度、双读、回退、escape hatch。
Trace 泄露敏感信息	可观测系统变成数据泄露面	脱敏、HMAC、访问控制、Trace/Audit 分离。

21.1 反模式清单

- 把缓存命中当成授权通过。
- 让 Router 直接拥有所有业务字段和权限逻辑。
- 让领域事件写死前端字段路径。
- 让 Event Mapping condition 执行任意脚本。
- 让 private-high 字段进入 IndexedDB。
- 无 cursor 签名地暴露分页游标。
- 不提供 SDK 类型，要求业务手写 DTO。
- 一次性迁移所有旧接口。
- 第一版同时做 Query、Command、Outbox、跨站授权。
- 生产 trace 记录字段原始值或完整 payload。

22. 最终建议

Formal V9 的核心调整是把平台从“架构能力完整”进一步收敛到“工程可采用”。

最终建议:

第一阶段:

SDK Lite Mode + View + Query + Domain Node 授权 + 安全缓存 + 标准响应。

第二阶段:

Contract Registry + Event Mapping + LKG + Emergency Revocation + Legacy Migration。

第三阶段:

Command Plane + Outbox + 命令审计 + 写侧驱动读侧失效。

未来阶段:

Cross-Site Consent & Delegation Plane。

关键成功条件不是平台有多少抽象，而是:

- 业务开发能轻松声明和查询数据。
- IDE 和类型系统能减少认知负担。
- 缓存不会成为隐性授权通道。
- 列表、分页、聚合能覆盖真实页面需求。
- 事件失效不污染领域事件。
- 老接口能渐进迁移。
- 失败和部分响应有稳定协议。
- Trace 和 Audit 不成为新的敏感数据面。

一句话总结:

> 该平台应先成为一个可被业务接受的、安全的只读数据闭环，再逐步扩展为完整的联邦式领域数据与命令平台。

A. 附录: 契约与协议模板

A.1 Field Contract

```
field: me.profile.email
ownerDomain: user
type: string
nullable: true
sensitivity: private-high
permission:
  read: user.profile.email.read
cache:
  scope: private
  ttl: 60s
  persistent: false
failure:
  mode: error-if-denied
versions:
  contractVersion: contract_v12
thirdParty:
  shareable: true
  requiresConsent: true
  allowedPurposes: [account_linking]
  defaultRetention: 7d
```

A.2 Collection Contract

```
resource: project.list
ownerDomain: project
type: connection
pagination:
  mode: cursor
  maxPageSize: 50
  cursor:
    signed: true
    expiresIn: 30m
    binds:
      - canonicalFilterHash
      - canonicalOrderHash
      - authContextFingerprint
      - contractVersion
allowedFilters:
  status:
    operators: [eq, in]
allowedOrderBy: [updatedAt, createdAt, name]
allowedExpands:
  owner:
    fields: [id, name, avatar]
count:
  visibleCount:
    mode: optional
    precision: exact_or_estimated
totalCount:
```

```
permission: project.admin.readTotalCount
```

A.3 Response Envelope

```
{
  "data": {},
  "errors": [],
  "meta": {
    "requestId": "req_123",
    "traceId": "trace_abc",
    "partial": false,
    "contractVersion": "contract_v12",
    "policyVersion": "policy_v9",
    "elapsedMs": 42
  }
}
```

A.4 Event Mapping DSL 示例

```
condition:
  op: and
  args:
    - op: eq
      left: tenantId
      right: "${event.tenantId}"
    - op: intersects
      left: changedAttributes
      right: [displayName, avatarFileId]
```

A.5 Emergency Revocation 示例

```
{
  "revocationId": "rev_001",
  "type": "policy-revoke",
  "minPolicyVersion": "policy_v11",
  "fields": ["billing.invoice.latest"],
  "sensitivity": ["private-high", "secret"],
  "action": "fail-closed",
  "signature": "sig_rev"
}
```

A.6 SDK Lite 示例

```
const result = await data.query({
  me: { profile: ["name", "avatar"] },
  global: { config: ["appName"] }
})

if (result.errors.length > 0) {
  sdk.handleErrors(result.errors)
}
```

A.7 definePageData 示例

```
export const pageData = definePageData({
```

```
critical: {
  me: { profile: ["name", "avatar"] }
},
lazy: {
  project: {
    list: collection({ first: 20 }).select({
      fields: ["id", "name"],
      expand: {
        owner: ["id", "name"]
      }
    })
  }
}
})
```

A.8 PoC 必跑测试清单

字段未注册默认拒绝。

Router 放行但 Domain Node 拒绝。

权限降级后旧缓存不可用。

租户切换后 private/tenant cache 不串。

高敏字段不进入 IndexedDB。

totalCount 无权限不返回。

Event Mapping 缺失时高敏缓存 fail-closed。

LKG 过期后 private-high fail-closed。

Emergency Revocation 覆盖未过期 LKG。

cursor 被篡改时拒绝。

partial response 前端能稳定渲染。

SDK Lite 模式可独立使用。

definePageData 拼错字段能在本地发现。